



Bansilal Ramnath Agarwal Charitable Trust's
Vishwakarma Institute of Information Technology, Pune-48
(An Autonomous Institute affiliated to Savitribai Phule Pune University)
Department of Information Technology

Final Year Project Work Guidelines B.Tech.
(Information Technology)
2020R1 Course
(With effect from Academic Year 2025-26)

Project Title	:	Secure and Extensible Remote System Administration via Large Language Model Agents using the Model Context Protocol
Project Area	:	Generative AI, Natural Language Processing and Remote Access
Internal Guide	:	Prof. Hanmant Magar



Bansilal Ramnath Agarwal Charitable Trust's
Vishwakarma Institute of Information Technology, Pune-48
(An Autonomous Institute affiliated to Savitribai Phule Pune University)
Department of Information Technology

SYLLABUS

Project Work (CAUA40204)

Examination Scheme:

Continuous evaluation: 100 Marks

PR/OR:50 Marks

Course Objectives:

1. To apply SDLC and meet the objectives of proposed development or research work
2. To test rigorously before deployment of work in objective 1
3. To validate the work undertaken during objective 1 and 2
4. To consolidate the development or research work as a project report.

Course Outcomes:

On completion of the course, student will be able to–

1. Produce evidence of independent investigation
2. Analyze the results and their interpretation intensively and critically.
3. Report and present the original results in an orderly way and placing the open questions in the right perspective.
4. Link techniques and results from literature as well as actual research and future research lines with the research.
5. Appreciate practical implications and constraints of the specialist subject

Guidelines

The student shall complete the work of the Project which will consist of problem statement, literature review, SRS, Model and Design, Selection of Technology and Tools, Installations, UML implementations, testing, Results, performance discussions using data tables per parameter considered for the improvement with existing/known algorithms/systems and comparative analysis and validation of results and conclusions. The candidate shall deliver a presentation on the advancement in Technology pertaining to the selected project topic. The examinee will be assessed by a panel of examiners of which one is necessarily an external examiner. The assessment will be broadly based on work undergone, content delivery, presentation skills, documentation, question- answers and report. The student shall prepare and submit the report of Project work in standard format for satisfactory completion of the work that is duly certified by the concerned guide and head of the Department/Institute.



Bansilal Ramnath Agarwal Charitable Trust's
Vishwakarma Institute of Information Technology, Pune-48
(An Autonomous Institute affiliated to Savitribai Phule Pune University)
Department of Information Technology

Index

Sr. No.	Content	Page No.
1	Certificate	5
2	Acknowledgement	6
3	Undertaking by Students	7
4	Project Work Schedule	8
5	Group Details	9
6	Abstract	10
7	List of Figures	11
8	List of Tables	12
9	Chapter 1: Introduction	13
9.1	Introduction	13
9.2	Problem Statement	14
9.3	Scope of Research	15
9.4	Research Hypothesis	16
9.5	Objectives	17
9.6	Organization of the Report	17
10	Chapter 2: Literature Review	19
10.1	Background	19
10.2	Review of Existing Work	20
10.3	Literature Survey Table	21
10.4	Summary of Literature Review & Research Gap	23
11	Chapter 3: Methodology	26
11.1	Materials	26



Bansilal Ramnath Agarwal Charitable Trust's
Vishwakarma Institute of Information Technology, Pune-48
(An Autonomous Institute affiliated to Savitribai Phule Pune University)
Department of Information Technology

11.2	Methodological Approach	27
11.3	Algorithmic Framework	27
11.4	System Architecture	28
11.5	Use Case Model	29
11.6	Systemic Workflow	29
11.7	Evaluation Method	32
11.8	Ethical & Practical Considerations	33
11.9	Summary of Methodology	33
12	Chapter 4: Results and Discussion	35
12.1	Summary of Results and Discussion	35
13	Chapter 5: Conclusion and Recommendations	41
13.1	Conclusion	41
13.2	Recommendations and Future Work	42
14	References	44



Bansilal Ramnath Agarwal Charitable Trust's
Vishwakarma Institute of Information Technology, Pune-48
(An Autonomous Institute affiliated to Savitribai Phule Pune University)
Department of Information Technology

CERTIFICATE

This is to certify that the project entitled **Secure and Extensible Remote System Administration via Large Language Model Agents using the Model Context Protocol** has been successfully completed by:

Roll No.	GR. No.	Division	Name of Student
431018	22210766	A	Arya Jalindar Kadam
432065	22210270	B	Tejaswini Jaywant Durge
431063	22211060	A	Soham Atul Deshmukh
431064	22210330	A	Prasad Rajaram Kute

In partial fulfillment of the requirements for the award of the degree of **Bachelor of Technology (B.Tech.) in Information Technology** From **Vishwakarma Institute of Information Technology**, Pune for the academic year 2025–2026.

This work has been carried out under the guidance of Prof. Hanmant Magar, Department of Information Technology.

It is certified that this project is the students' original work and has not been submitted, either in part or full, for the award of any other degree or diploma.

Date: 4/05/2026

Prof. Hanmant Magar

Project Guide

Prof. Pravin.Futane

Head, IT Department

Prof. Vivek Deshpande

Director, VIIT, Pune



Bansilal Ramnath Agarwal Charitable Trust's
Vishwakarma Institute of Information Technology, Pune-48
(An Autonomous Institute affiliated to Savitribai Phule Pune University)
Department of Information Technology

ACKNOWLEDGEMENT

We express our sincere gratitude to the **Department of Information Technology, Vishwakarma Institute of Information Technology, Pune**, for providing us with the opportunity to carry out this project.

We heartfully thank our internal guide, **Prof. Hanmant Magar**, for their valuable guidance, constant support, and insightful feedback throughout the project. We take this opportunity to express our sincere gratitude to **Prof. Pravin Futane, Head of the Department**. We also thank our Project Coordinator and all faculty members for their encouragement and academic assistance.

We are grateful to our classmates, laboratory staff, and everyone who helped us directly or indirectly during the development of this work. Lastly, we thank our families for their continuous motivation and support.

1. **Arya Kadam**
2. **Tejaswini Durge**
3. **Soham Deshmukh**
4. **Prasad Kute**



Bansilal Ramnath Agarwal Charitable Trust's
Vishwakarma Institute of Information Technology, Pune-48
(An Autonomous Institute affiliated to Savitribai Phule Pune University)
Department of Information Technology

UNDERTAKING BY STUDENTS

We, the students of BTech Information Technology (IT), hereby assure that we will follow all the rules and regulations of VIIT related to the project work for the academic year **2025-26**. The Project entitled **Secure and Extensible Remote System Administration via Large Language Model Agents using the Model Context Protocol** will be fully designed and developed by us and no part of the project/ full project will be designed and developed by any external entity or copied from some external resources. We are fully aware that copying or taking help of any external agency in the development of our project is totally unethical and illegal. The examiners have / University has full rights to initiate an action against us as per university norms if involved in unfair/ illegal/ unethical work.

Sr. No.	GR. No.	Roll No.	Name of Student	Signature
1	22210766	431018	Arya Jalindar Kadam	
2	22210270	432065	Tejaswini Jaywant Durge	
3	22211060	431063	Soham Atul Deshmukh	
4	22210330	431064	Prasad Rajaram Kute	



Bansilal Ramnath Agarwal Charitable Trust's
Vishwakarma Institute of Information Technology, Pune-48
(An Autonomous Institute affiliated to Savitribai Phule Pune University)
Department of Information Technology

PROJECT WORK SCHEDULE

Sr. No.		Date
1.	Registration of Project Groups	Week 1
2.	Allocation of Guide and Project Topic Submission	Week 2
3.	Submission of Abstract to Project Guide and Project Coordinator in the Prescribed Format	Week 3
4.	Project Review – 0 (For topic and scope finalization)	Week 4
5.	Project Review - I	Week 7
6.	Project Review - II	Week 10
7.	Verification of Project Work Book By Internal Guide (before submission of Preliminary Project Report)	Week 11
8.	Submission of Final Preliminary Project Report in Prescribed Format	Week 12
	External Term-work Evaluation	As per Institute Schedule



Bansilal Ramnath Agarwal Charitable Trust's
Vishwakarma Institute of Information Technology, Pune-48
(An Autonomous Institute affiliated to Savitribai Phule Pune University)
Department of Information Technology

BTECH PROJECT GROUP DETAILS
(Academic Year:2025-26 Sem:08)

Project Group No.: 06

Project Title: Secure and Extensible Remote System Administration via Large Language Model Agents
using the Model Context Protocol

SrNo.	Roll No.	Student Name	Mobile No.	Email id	T. E. Result
1	431018	Arya Kadam	8779831758	<i>arya.22210766@viit.ac.in</i>	
2	432065	Tejaswini Durge	8446528408	<i>tejaswini.22210270@viit.ac.in</i>	
3	431063	Soham Deshmukh	8668466550	<i>soham.22211060@viit.ac.in</i>	
4	431064	Prasad Kute	7558750366	<i>prasad.22210330@viit.ac.in</i>	

Name & Signature Internal Guide

Signature of Head of
Department

Name & Signature of External Guide



Bansilal Ramnath Agarwal Charitable Trust's
Vishwakarma Institute of Information Technology, Pune-48
(An Autonomous Institute affiliated to Savitribai Phule Pune University)
Department of Information Technology

Abstract

The developed system allows remote management of computers using simple text messages sent through Telegram. At its core, a large language model (LLM) agent interprets these plain-language commands. An LLM agent sits behind the chat, figures out what the user wants, and executes it on the host through a local FastMCP server. The Model Context Protocol (MCP) ties these pieces together that gives a structured, schemadirect linkage of these elements by doing away with the need for numerous custom integrations. Knowing that it is dangerous to entrust an AI to do something, shell commands, the solution that is implemented is composed of a robust three-tier safety system. To begin with, a per-user whitelist of permissions, protects the issuing of commands by authorized people. Second, a blacklist is a regular expression based blacklist that proactively identifies and prevents possible harmful commands, like 'rm -rf', before they reach the operating system. Finally, a pattern filter scrutinizes the LLM-generated Python code and prevents the inclusion of dangerous imports. The proposed solution agent was constructed using LangGraph and has been successfully tested with more than ten different models from major providers like Google, OpenAI, and Anthropic. Notably, switching between these AI models did not require modifications to the system's toolset. In performance evaluations, the system achieved a 92 % success rate in 50 administrative tasks, effectively neutralized all 30 simulated adversarial attacks and delivered responses in an average of less than four seconds. This system has been in continuous operation since early 2026.



Bansilal Ramnath Agarwal Charitable Trust's
Vishwakarma Institute of Information Technology, Pune-48
(An Autonomous Institute affiliated to Savitribai Phule Pune University)
Department of Information Technology

LIST OF FIGURES

Figure No.	Title
Figure 3.1	Sandbox Validation Pipeline
Figure 3.2	System Architecture
Figure 3.3	Use Case Diagram
Figure 3.4	Data Flow Diagram
Figure 3.5	Coding Mode State Machine Diagram



Bansilal Ramnath Agarwal Charitable Trust's
Vishwakarma Institute of Information Technology, Pune-48
(An Autonomous Institute affiliated to Savitribai Phule Pune University)
Department of Information Technology

LIST OF TABLES

Table No.	Title
Table 3.1	Summary of Materials
Table 3.2	Evaluation Metrics
Table 4.1	Functional Test Results by Task Category
Table 4.2	Adversarial Test Results (30 Payloads)



CHAPTER 1: INTRODUCTION

1.1 Introduction

Anyone who has managed a computer remotely knows the drill. You open an SSH session or fire up PowerShell Remoting, and you type precise commands from memory. Need to check what process is hogging your RAM? You had better remember the exact syntax for tasklist or top or Get-Process with the right flags. Remote Desktop gives you a graphical view, but it is sluggish over poor connections and still expects you to navigate menus and type commands manually. VNC is similar. These tools work, nobody disputes that, but they all share the same underlying assumption: the person on the other end already knows what to do and how to tell the machine to do it. If you forget a flag, misspell a path, or confuse the syntax between Linux and Windows, you are stuck. There is no room for saying "just show me what is using all the memory" and having the machine figure out what you actually mean.

Large Language Models changed that equation. Models like GPT-4, Gemini, and Claude can read a vague English sentence, figure out which operating system commands would answer the question, and even chain multiple steps together when one command depends on the output of another. A user who types "show me the ten biggest files in my downloads folder" does not need to know whether they are on Windows or Linux, does not need to recall the flags for sort or du or Get-ChildItem. The model handles all of that. It picks the right command, runs it, reads the output, and explains the result in plain language. That is genuinely useful for people who are not command-line experts but still need to get things done on their machines while away from the desk.

But plugging an LLM directly into a live operating system creates serious problems, and we ran into all of them during development.

The first problem is what people in the field call the "M x N" integration bottleneck. There are M different model providers out there, OpenAI, Google, Anthropic, various open-source options, and each one has its own calling conventions, its own way of describing tools, its own quirks around function parameters. On the other side, there are N different things you might

want the model to do: run shell commands, read files, write files, launch applications, query system metrics, execute Python scripts, and so on. Without some shared standard, you end up writing custom glue code between every model and every tool. We experienced this firsthand. Our early prototype required changes to three files and two process restarts just to expose a single new capability. When you multiply models by tools, the maintenance cost grows fast and the whole thing becomes brittle. One API update from a provider can break everything downstream.

The second problem is security, and this one is harder. An LLM does not actually understand what a shell command does to the file system. It predicts tokens. It can hallucinate commands that look plausible but cause damage. Worse, it can be tricked. Prompt injection is a well-documented attack where malicious input is crafted to make the model ignore its instructions and execute something dangerous instead. If your system hands an LLM the ability to call subprocesses and run arbitrary strings on the host, you have effectively given a statistical text generator root access to your machine. The OWASP Foundation now ranks prompt injection as the number one risk for LLM applications. During our own testing, we saw the model attempt to cat our .env file to "check the configuration," which would have leaked every API key we had. You cannot deploy something like this without multiple layers of filtering between the model's output and the operating system kernel.

The third problem is access. If you need a VPN, a dedicated client, and a multi-step authentication ceremony just to run a quick command on your home machine, most people will not bother. The whole point of remote administration is convenience: you are on your phone at a coffee shop and you want to check if a service crashed, or you want to kill a runaway process before it eats your disk. The interface needs to be something people already have installed, something that sends notifications, something that works on every platform without setup.

For the integration bottleneck, we adopted Anthropic's Model Context Protocol, or MCP. It is an open standard released in late 2024 that works like a universal adapter between LLMs and tools. A tool publishes its name, a description, and a JSON Schema describing its arguments. Any MCP-compliant model can read that schema and call the tool correctly. We built a local



Bansilal Ramnath Agarwal Charitable Trust's
Vishwakarma Institute of Information Technology, Pune-48
(An Autonomous Institute affiliated to Savitribai Phule Pune University)
Department of Information Technology

FastMCP server that exposes our five tool categories through these schemas. The result is that adding a new tool means writing one Python class and restarting the server. No changes to the agent logic, no changes to the Telegram interface, no per-model adapters. We have tested the same tool layer with six Gemini models, four OpenAI models, and Anthropic Claude without modifying a single line in the tool code. That is the $M + N$ reduction that MCP promises, and we can confirm it works in practice.

The broader point of this project is to demonstrate that giving an LLM agent control over a real operating system does not have to be reckless. With the right protocol for tool standardization, the right layering of security checks, and a sensible choice of user interface, it is possible to build something that is both genuinely useful for day-to-day administration and resistant to the failure modes that make autonomous AI execution dangerous. We have been running OrbitOS on personal machines since early 2026. It handles routine tasks reliably, it has stopped every adversarial test we have designed, and switching between LLM providers takes a single config change. It is not perfect, the limitations are real and we discuss them later, but it works well enough to use every day.

1.2 Problem Statement

Remote system administration today is stuck between two worlds. Traditional tools like SSH and RDP give full control but demand expert-level syntax knowledge. LLMs can interpret plain English, but connecting them to a live operating system safely remains an unsolved engineering challenge.

1. Fragmented Tool Integration (The $M \times N$ Problem)

There is no shared standard for how an LLM discovers and invokes system tools.

- **The Issue:** Each model provider has its own function-calling convention. Every new tool needs a separate adapter for every provider.
- **The Burden:** Maintaining M providers times N tools kills velocity. One API update from any provider can break the whole chain.



Bansilal Ramnath Agarwal Charitable Trust's
Vishwakarma Institute of Information Technology, Pune-48
(An Autonomous Institute affiliated to Savitribai Phule Pune University)
Department of Information Technology

2. Uncontrolled Execution Risk

An LLM wired to a subprocess call is a probabilistic text generator, not a sysadmin. It predicts tokens; it doesn't understand the physical reality of a disk wipe.

- **The Threat:** Prompt injection is now the top OWASP risk for LLM applications.
- **The Gap:** Most frameworks lack layered filtering. During testing, models may attempt to "helpfully" read sensitive .env files unprompted. Security is currently left as an exercise for the developer.

3. Infrastructure-Heavy Remote Access

Traditional methods are heavy and restrictive:

- **SSH/RDP:** Need key management, VPNs, or port forwarding.
- **WinRM:** Requires complex certificate setups.
- **The Problem:** None of these work well from a phone or a coffee shop. Admins shouldn't have to configure firewall rules just to kill a runaway process on the go.

4. Rigid, Hard-Coded Tool Sets

Most agent systems bake capabilities into core logic.

- **The Friction:** Adding a tool means modifying the agent, rewriting prompts, and adding parsing code.
- **The Goal:** A system needs a drop-in mechanism where writing one class allows the LLM to pick it up automatically.

5. Single-Provider Lock-In

Systems tied to one provider break during outages or price hikes.

- **The Solution:** Swapping from Gemini to GPT to Claude should be a simple configuration change, not a total rewrite of the tool layer.



1.3 Scope of Research

The focus of this B.Tech final year project is the creation, development, and rigorous testing of a remote PC control agent capable of operating via natural language through a mobile interface. The implementation and defined boundaries of the project are detailed below:

1. MCP-Based Tool Layer Development

We developed five core tool classes—**Shell, File Management, Python Execution, App Launcher, and System Monitor**—utilizing Anthropic's **Model Context Protocol (MCP)** via the FastMCP library.

- **Objective:** Standardizing JSON Schema definitions to ensure any compliant model can invoke tools without per-model "glue code."
- **Boundary:** We adopted an existing open standard (MCP) rather than creating a proprietary protocol.

2. Telegram Interface Integration

The user-facing component was built entirely as a **Telegram bot**, serving as the primary control hub.

- **Features:** Authentication, session tracking, command routing, file transfers, and inline keyboards for model switching.
- **Boundary:** The project focused exclusively on Telegram; while the architecture supports web or desktop GUIs, they were not implemented in this phase.

3. Three-Layer Sandbox Design and Testing

To address execution risks, we engineered a security framework consisting of:

- **Identity-Based Access Control:** Three permission tiers (Read-only, User, Admin).



Bansilal Ramnath Agarwal Charitable Trust's
Vishwakarma Institute of Information Technology, Pune-48
(An Autonomous Institute affiliated to Savitribai Phule Pune University)
Department of Information Technology

- **Command Filtering:** A regex blacklist for shell commands and pattern-based filters for Python.
- **Data Protection:** Triple-checkpoint `.env` file blocking and "Human-in-the-loop" confirmations for deletions.
- **Boundary:** Filtering was implemented at the host level. Full containerized isolation or micro-VM sandboxing is categorized as future work.

4. LangGraph ReAct Agent Implementation

We transitioned from a CLI wrapper to a dynamic **LangGraph agent** that utilizes a "Reason-Act" loop.

- **Functionality:** Maintains a 30-message sliding window for conversation history and supports runtime switching between **Google Gemini, OpenAI, and Anthropic**.
- **Boundary:** The project utilized off-the-shelf commercial APIs; no custom model fine-tuning was performed.

5. Structured "Coding Mode"

Beyond simple commands, we implemented a multi-turn workflow for software development tasks following an **Explore → Plan → Execute → Verify** sequence.

- **Technical Detail:** This involved a custom state machine in the Telegram bridge and a dedicated system prompt.
- **Boundary:** Designed for functional remote editing rather than replacing full-featured IDEs.

6. Functional and Adversarial Evaluation

The system was rigorously evaluated on a Windows 11 environment using **Claude 3.5 Sonnet** as the primary model.

- **Legitimate Tasks:** 50 administration tasks (Networking, Process Control, etc.).



Bansilal Ramnath Agarwal Charitable Trust's
Vishwakarma Institute of Information Technology, Pune-48
(An Autonomous Institute affiliated to Savitribai Phule Pune University)
Department of Information Technology

- **Adversarial Tests:** 30 attack payloads covering prompt injection and privilege escalation.
- **Boundary:** Evaluations focused on success rates and security layer efficacy rather than comparative benchmarking against existing admin systems.

7. Cross-Platform and Multi-Model Validation

Final validation involved deploying OrbitOS on **macOS** and rotating between different models, such as **Gemini 2.5 Flash**, for daily personal use.

1. **Objective:** Confirming that the tool layer and sandbox remain stable regardless of the underlying OS or LLM provider.
2. **Boundary:** Limited to Windows and macOS; exhaustive testing across all Linux distributions was not within the current project scope.

1.4 Research Hypothesis

We started this project with a bet: that you can take a commercially available LLM, wire it to a real operating system through a standardized protocol like MCP, put a three-layer filter between its output and the kernel, and end up with something that is both useful enough to replace manual SSH sessions for everyday tasks and safe enough that it will not trash your machine when the model hallucinates or gets fed a malicious prompt. More specifically, we expected that the MCP abstraction would let us swap between models from different providers without touching the tool code, that the regex command blacklist and pattern-based Python filter would catch destructive commands before they execute, and that Telegram would work as a practical daily-driver interface without the infrastructure overhead of VPNs or dedicated clients.

The counter-argument is obvious: regex blacklists are bypassable, string-matching import filters miss creative workarounds like `importlib`, and running on the bare host without a container or micro-VM means one missed pattern could be catastrophic. We did not claim the sandbox is unbreakable. The hypothesis was narrower than that. We wanted to show that for the class of

attacks an LLM is likely to generate, whether through hallucination or through standard prompt injection techniques documented in current literature, a lightweight host-level filter is effective enough to be practical while being simple enough to actually ship and maintain. If a determined human attacker with direct shell access wanted to bypass it, they probably could. But that is a different threat model from an LLM accidentally running a destructive command because it misunderstood the user's request.

1.5 Objectives

- 1. Build a Provider-Agnostic LLM Agent using MCP:** Implement a FastMCP server that exposes system administration tools through JSON Schema definitions. The goal is to allow any compliant model (Gemini, GPT, Claude) to discover and invoke tools without per-model adapter code. This is validated by running the tool layer across ten-plus models from three different providers without modification.
- 2. Develop a Five-Tool Administrative Capability Suite:** Create a comprehensive toolset consisting of **ShellTool** (OS command execution), **FilesTool** (file management and search), **PythonExecTool** (isolated script execution), **AppTool** (cross-platform app launching), and **SystemTool** (resource monitoring via psutil). Each tool is a Python class extending a common BaseTool interface.
- 3. Design and Implement a Three-Layer Execution Sandbox: * Layer 1:** Identity-based access control with three permission tiers (Read-only, User, Admin).
 - a. **Layer 2:** A regex blacklist to intercept destructive shell commands before subprocess spawning.
 - b. **Layer 3:** A pattern-based Python filter to block dangerous imports and builtin calls.
 - c. **Safety Backstop:** Implement `.env` hard-blocking at three separate checkpoints to protect sensitive credentials.
- 4. Build a Telegram-Based UI with Zero Infrastructure Overhead:** Implement a full-featured Telegram bot that handles authentication, session management, natural

language routing, inline model switching, and file transfers (up to 50MB). The objective is to allow a user to administer their machine from a phone without configuring VPNs, SSH keys, or port forwarding.

- 5. Implement a Structured Coding Mode for Development Tasks:** Build a state machine (**NORMAL** → **WAITING_DIR** → **WAITING_GOAL** → **CODING**) that directs the agent through a disciplined **Explore** → **Plan** → **Execute** → **Verify** workflow. This mode supports multi-file editing with extended timeouts and real-time progress streaming.
- 6. Validate Functionality and Security Through Structured Testing:** Execute **50 legitimate administration tasks** across five categories to measure success rates and response times. Perform **30 adversarial tests** covering command injection, prompt injection, and privilege escalation to ensure the sandbox prevents unauthorized access to the operating system.

1.6 Organization of the Report

This report is structured as follows:

The remainder of this report is organized into the following chapters:

- **Chapter 2: Literature Review**

This chapter traces the three primary bodies of prior work that shaped OrbitOS:

- **LLM Agent Research:** An analysis of models performing system-level tasks.
- **Model Context Protocol (MCP):** A comparison of MCP against earlier tool-binding approaches such as ToolFormer and Gorilla.
- **Sandboxing Literature:** A review of isolation technologies ranging from Docker containers to Firecracker micro-VMs.
- **The Research Gap:** The chapter concludes with a summary table identifying the specific gaps in current literature that OrbitOS aims to fill.

- **Chapter 3: Methodology**

This chapter details the technical implementation and development process:

- **System Architecture:** Descriptions of the five-tool suite, the FastMCP server, and the LangGraph ReAct agent's context management.
- **Security Framework:** The reasoning behind utilizing host-level filtering over containerized isolation in the three-layer sandbox.
- **Interface Design:** The logic behind the Telegram bridge, the state machine, and the multi-turn Coding Mode workflow.
- **Evaluation Strategy:** The methodology for designing and scoring the 50 functional tasks and 30 adversarial payloads.

- **Chapter 4: Results and Discussion**

This chapter presents a quantitative and qualitative analysis of the system's performance:

- **Functional Performance:** A 92% success rate (46/50 tasks) with an average response time of 3.0 seconds.
- **Security Efficacy:** A 100% block rate across all 30 adversarial payloads.
- **Analysis:** A breakdown of the four functional failures and a study of system latency (1.8s LLM API vs. 200ms MCP server) to identify future optimization targets.

- **Chapter 5: Conclusion and Recommendations**

The final chapter summarizes the project's contributions and future trajectory:

- **Key Findings:** Validation of MCP as a solution to the \$M \times N\$ integration problem and the viability of Telegram as a zero-infrastructure admin interface.
- **Limitations:** An honest assessment of the risks inherent in regex-based filtering and the lack of hardware-level isolation.
- **Future Work:** Proposed enhancements including Firecracker micro-VM isolation, a community tool marketplace, and a dedicated web-based dashboard.



CHAPTER 2: LITERATURE REVIEW

2.1 Background

Remote system administration has been a solved problem for decades, in the narrow sense that the tools exist and they work. SSH has been stable since the late 1990s. Remote Desktop Protocol shipped with Windows 2000. PowerShell Remoting became standard with Windows Server 2008. These tools give administrators full control over remote machines, and the underlying protocols are mature, well-documented, and battle-tested. Nobody questions whether they work technically.

The problem is that they were all designed with the same assumption baked in: the person on the other end is an expert. SSH gives you a shell. That shell will do exactly what you type. If you type the wrong thing, the shell will do that too, efficiently and without complaint. The tools have no concept of intent. They cannot distinguish between a junior developer who meant to delete temporary files and accidentally typed the wrong path, or a tired administrator who ran a command in the wrong terminal window. The interface is a direct wire to the operating system, and human error is entirely the user's responsibility.

Large Language Models introduced something different. Starting roughly with the GPT-3 generation and accelerating sharply with GPT-4, Gemini, and Claude, it became clear that these models could do more than answer trivia questions. They could read a vague English sentence, figure out which shell command it corresponded to, and even explain their reasoning. "Show me which process is using the most CPU" is not a shell command, but a capable LLM can translate it into the right invocation of `ps`, `top`, or `Get-Process` depending on the operating system, run it, parse the output, and respond in plain English. That is a qualitatively different kind of interface from anything that existed before.

The research community noticed. By 2023, a wave of papers had appeared exploring whether you could give LLMs actual tool access, not just the ability to describe what commands to run, but the ability to actually run them and loop on the results. The ReAct

framework from Yao et al. gave this a name and a formal structure. ToolFormer from Schick et al. and Gorilla from Patil et al. explored how to train models to invoke APIs reliably. Experiments like the one from Cao et al., where GPT-4 was pointed at 150 real Linux administration tasks and handled the majority of them autonomously, demonstrated that the reasoning capability was clearly there for system-level work specifically.

But every one of these systems ran into the same two problems when you tried to push them beyond controlled experiments. The first is the integration problem. Each model provider had its own function-calling convention, its own way of describing what tools are available, its own quirks about argument formats. A tool that worked with GPT-4 needed a completely different adapter for Gemini, and another one for Claude. As the number of tools and providers grew, the adapter matrix grew with it. The second is the safety problem. An LLM with tool access is not reasoning about consequences the way a human would. It is selecting tokens that are statistically likely given the context. That means it can hallucinate plausible-looking but destructive commands, and it can be manipulated by prompt injection into running commands that a malicious input was specifically crafted to trigger.

These two problems, the integration bottleneck and the execution safety question, form the backdrop against which OrbitOS was designed. The sections that follow trace the prior work that directly influenced specific decisions we made during development.

2.2 Review of Existing Work

The literature on LLM-enabled tooling falls into two roughly parallel veins: research that focuses on teaching or prompting models to call tools reliably, and work that focuses on the safety and isolation problems that arise when models get control over I/O or execution. On the first front, approaches like ToolFormer and Gorilla explored training-time and prompting techniques to nudge models into invoking external APIs rather than hallucinating. The ReAct family of methods reframed the interaction as a loop—reason, act (call a tool), observe, and repeat—and that pattern became the operational model for many subsequent systems that aim to

close the loop between language and action.

Practical experiments showed both the promise and the limits. Cao et al.'s study that asked GPT-4 to perform 150 real Linux administration tasks provided early evidence that general-purpose LLMs can sequence system commands and produce useful outcomes. At the same time, plugin-style systems (e.g., early ChatGPT plugins) and platform studies (Iqbal et al.) revealed a high surface area for data leakage and improper authorisation when tools are poorly constrained.

On the protocol and integration side, the community has moved from ad-hoc adapters toward wire-level standards. The Model Context Protocol (MCP) is an explicit response to the $M \times N$ problem: tools publish JSON Schemas for their arguments, and MCP-compliant clients and models can exchange typed requests without bespoke per-model glue. MCP follows the same spirit as the Language Server Protocol's separation of concerns and addresses a long-standing engineering pain point in tool integration.

The safety literature is extensive and relevant. Prompt injection research from Liu et al. and follow-ups showed how models can be coaxed into ignoring constraints; Greshake and others demonstrated how retrieval and document ingestion can reintroduce adversarial instructions. OWASP's recent attention to LLM risks crystallised many of these findings into practical mitigation recommendations. In parallel, systems research has evaluated isolation approaches: lightweight host filtering (regexes and static import checks), container-based isolation (Docker), memory-safe execution (WebAssembly), and micro-VM approaches (Firecracker). Each option represents a trade-off between operational complexity and attack surface.

Agent frameworks and orchestration tools (LangChain, LangGraph, and similar graph-based libraries) provided sensible scaffolding for building ReAct-style agents and for wiring tool invocation into larger workflows. They make it straightforward to implement per-user contexts, structured tools, and execution graphs—features we relied on when

integrating MCP-backed tools into our LangGraph agent.

Taken together, the existing work provides the ingredients but not the full recipe. Prior work shows LLMs can act usefully, that typed tool interfaces reduce parsing errors, and that isolation reduces risk—but very few productions have combined a provider-agnostic protocol, a practical messaging-first UI, and a layered host-side sandbox in a single, daily-use system. That gap is where OrbitOS sits: we adopt MCP to solve integration, use Telegram for zero-infrastructure access, and layer pragmatic host-side protections to reduce the most likely failure modes while keeping the system lightweight enough to run on personal hardware.

2.3 Summary of Literature Review and Research Gap

Across the literature we reviewed, three clear lessons emerge. First, LLMs have reached a level of practical utility for system tasks: experiments such as Cao et al.'s GPT-4 study, and numerous ReAct-based implementations, show that language models can plan, call tools, and loop on observations to complete multi-step admin jobs. Second, typed and schema-driven tool interfaces substantially reduce parsing errors and brittle prompt engineering; MCP and similar proposals directly address the engineering burden of per-model adapters. Third, safety remains the dominant practical obstacle: prompt injection, data exfiltration via careless tooling, and creative workarounds around simple string filters are recurring themes in the security literature.

The design space for isolation and safety is mature but trade-off heavy. Full virtualization or micro-VMs reduce risk at the cost of complexity and resource overhead; containers and WebAssembly sit in the middle; host-side filtering is light and low-friction but imperfect. Agent frameworks and orchestration libraries ease implementation, but they do not solve safety or integration by themselves.

Our research gap statement is concise: while prior work demonstrates each ingredient individually—capable LLMs, better tool interfaces, and multiple sandboxing strategies—there is little published work showing a single, practical system that (a) uses a provider-agnostic



Bansilal Ramnath Agarwal Charitable Trust's
Vishwakarma Institute of Information Technology, Pune-48
(An Autonomous Institute affiliated to Savitribai Phule Pune University)
Department of Information Technology

protocol to collapse the $M \times N$ adapter problem, (b) provides a zero-infrastructure messaging-first UI for everyday use, and (c) employs layered, host-side protections that are lightweight enough to run on personal hardware yet effective enough to stop common LLM-generated failure modes. OrbitOS aims to occupy precisely that gap and to report realistic operational results (functional tests, adversarial tests, latency breakdowns) so others can judge the trade-offs for themselves.

CHAPTER 3: METHODOLOGY

3.1 Materials

We utilized a practical toolchain and commodity test hardware to ensure the system remains reproducible on a standard student-grade laptop or modest desktop. The materials used during implementation and evaluation are detailed below.

3.1.1 Hardware and Operating Systems

- **Testbed Hardware:** Windows 11 Desktop (Intel Core i5-12400, 16 GB RAM, 512 GB SSD). This machine served as the primary environment for formal functional and adversarial testing.
- **Secondary Hardware:** A macOS laptop was used for day-to-day validation and to confirm cross-platform behavior.
- **Primary Operating System:** Windows 11.
- **Secondary Operating System:** macOS (latest stable release at the time of testing).

3.1.2 Language and Runtime Environments

- **Language:** Python 3.11.
- **Environment Management:** Virtual environments (venv) were used to ensure isolation, with a requirements.txt file driving reproducible installations.
- **Core Architecture:** Asynchronous I/O based on the asyncio library.

3.1.3 Core Libraries and Frameworks

- **fastmcp:** Local implementation of the Model Context Protocol to host JSON-Schema tool endpoints.
- **LangGraph / LangChain:** Used for ReAct-style agent scaffolding and structured tool wrappers.



Bansilal Ramnath Agarwal Charitable Trust's
Vishwakarma Institute of Information Technology, Pune-48
(An Autonomous Institute affiliated to Savitribai Phule Pune University)
Department of Information Technology

- **python-telegram-bot:** Serves as the bridge for Telegram message handling and user interface.
- **psutil:** Utilized for fetching system telemetry, including CPU, memory, disk usage, and process listings.
- **Pydantic:** Used for typed schemas and data validation for MCP tools.

3.1.4 LLM Providers and Models

The system was designed for provider-agnosticism. The following models were validated:

- **Google Gemini Family:** Gemini 2.5 Flash (default), Gemini 2.5 Pro (for Coding Mode).
- **OpenAI GPT-4 Variants:** GPT-4o, GPT-4o-mini, GPT-4.1-mini, and GPT-4.1-nano.
- **Anthropic Claude:** Validated for MCP support in specific validation runs.

3.1.5 Security and Testing Components

- **Security Suite:** Identity-based access control (Read-only, User, Admin) and a regex-based sandbox for shell and Python execution.
- **Test Corpus:** A custom dataset consisting of **50 functional tasks** (System Admin) and **30 adversarial payloads** (Security Testing).

Table 3.1 Summary of Materials

Item	Type	Version / Spec	Purpose
Test Machine	Hardware	Intel i5-12400, 16GB RAM, Win11	Platform for formal tests
Python	Runtime	3.11 (venv)	Application runtime

FastMCP	Protocol	Local Implementation	Host MCP tool endpoints
LangGraph / LangChain	Framework	Latest Stable	ReAct agent scaffolding
python-telegram-bot	Library	Latest Stable	Telegram interface bridge
psutil	Library	Latest Stable	System telemetry
Models Tested	LLMs	Gemini, GPT-4.x, Claude	Multi-model validation
Tool Classes	Code	Shell, Files, Python, App, System	Capabilities exposed via MCP
Test Corpus	Datasets	50 Functional / 30 Adversarial	System evaluation artifacts

3.2 Methodological Approach

This section explains how we built, validated, and measured OrbitOS. The approach is practical and iterative: start with a minimal MCP-backed toolset, add the LangGraph agent glue, then harden with layered sandboxing and measured tests.

1) Development workflow (high level):

- a) Prototype tools as Python classes extending a common BaseTool interface and a Pydantic schema.
- b) Host tools on a local FastMCP server so models can call them via JSON Schema.



Bansilal Ramnath Agarwal Charitable Trust's
Vishwakarma Institute of Information Technology, Pune-48
(An Autonomous Institute affiliated to Savitribai Phule Pune University)
Department of Information Technology

- c) Implement the LangGraph ReAct agent to select tools, invoke them, and loop on observations.
- d) Build the Telegram bridge for user I/O and session/state management.
- e) Harden with permission checks; iterate based on adversarial testing results.

2) System pipeline (runtime):

- a) User message arrives via Telegram → TelegramBridge authenticates and normalises input.
- b) If explicit shorthand (e.g., \$, /shell, >>>) route directly to FastMCP after sandbox checks; otherwise hand to the LangGraph agent.
- c) Agent reasons, fills MCP tool schemas, and issues JSON-RPC calls to FastMCP.
- d) FastMCP enforces auth + sandbox checks → executes tool → returns typed response.
- e) Agent observes results, decides next action or formats final response; results stream back to user through Telegram.

3) Tool design principles:

- a) Typed inputs: every tool exposes a JSON/Pydantic schema to eliminate brittle parsing.
- b) Short timeouts and iteration caps (120s default; 600s in Code Mode) to avoid stuck models.
- c) Output truncation and size limits to prevent accidental data leaks.
- d) Clear permission tiers per tool to map capabilities to readonly, user, or admin.

4) Agent behaviour and context management:

- a) Per-user UserContext with a 30-message sliding window; trim back to the initial HumanMessage to avoid orphaned tool calls.
- b) System prompt enforces ReAct loop behaviour and instructs the agent to prefer safe, minimal actions.
- c) Model switching via inline UI (/models) is runtime-configurable; Code Mode uses gemini-2.5-pro by default for larger context needs.

5) Sandbox validation and telemetry:

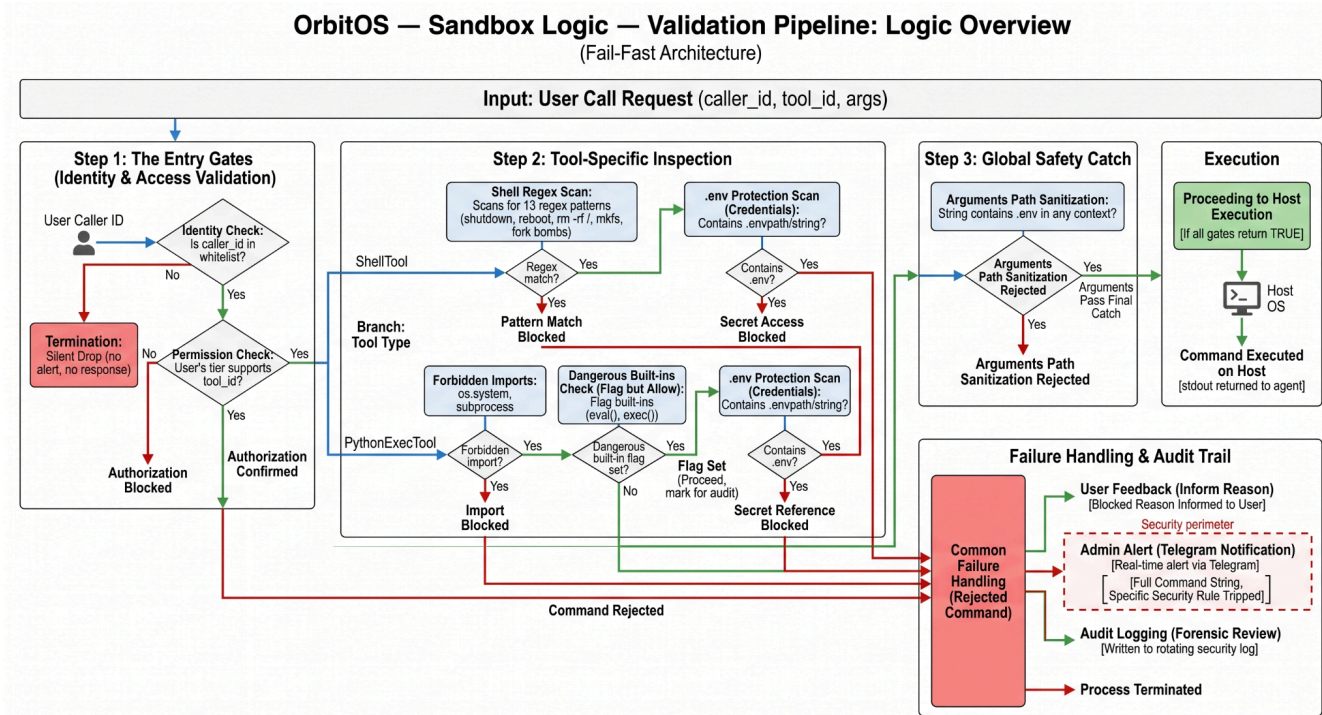


Figure 3.1: Sandbox Validation Pipeline— the three enforcement gates in sequence, showing where each class of attack gets caught before reaching the host OS.

- Three core enforcement gates (identity, regex blacklist, Python pattern scanner) run at FastMCP entry points.
- All rejected calls are logged and produce an admin alert in Telegram. Detailed logs include the requesting user_id, attempted tool, matched rule, and stack trace where applicable.
- We collected telemetry for every test run (timestamps, LLM latency, MCP processing time, tool execution duration) to build the latency breakdowns reported later.

6) Testing methodology:

- a) Functional tests: 50 hand-crafted tasks across five categories (file management, networking, process control, system info, Python scripting). Each task has an expected pass/fail criterion and a human-verified oracle.
- b) Adversarial tests: 30 payloads designed to probe prompt injection, command injection, import smuggling, and permission escalation.
- c) Each test run records: pass/fail, blocking layer (if any), end-to-end latency, LLM-only latency, MCP processing time, and any manual remediation steps.

Table 3.2 Evaluation Metrics

Metric	Definition	Observed (summary)
Functional success rate	% of 50 tasks that completed correctly	92% (46/50)
Adversarial block rate	% of 30 payloads blocked	100% (30/30)
Average response time	End-to-end median (seconds)	3.0 s
LLM contribution	Median time spent in external LLM API	1.8 s
MCP/host contribution	Median time in MCP + tool execution	< 0.2 s
Notable failures	Brief cause summary	4 functional failures — ambiguous prompts or edge-case CLI differences

3.3 Algorithmic Framework

The core algorithm in OrbitOS is the ReAct loop — Reason, Act, Observe — running inside a LangGraph execution graph. It is not a single function; it is a stateful graph where each node represents a decision point and edges represent transitions between thinking and doing. Here is how it actually runs.

Step 1 — Receive and parse the request. The message arrives from Telegram, already authenticated. The CommandRouter in checks for explicit shorthands first: a \$ prefix routes straight to ShellTool, a >>> prefix routes straight to PythonExecTool, and /shell, /files, /sys and their aliases each map to a specific tool and default action. If none of those patterns match, the message goes to the LangGraph agent as a natural language input.

Step 2 — Agent reasoning (the Reason node). The agent holds the current conversation in a UserContext object — up to 30 messages, trimmed back to the first HumanMessage when the window fills up to avoid Gemini's orphaned-tool-call errors. On each turn, the LLM sees the system prompt, the conversation history, and the list of available MCP tools with their JSON Schemas. It produces either a final text answer or a structured tool call specifying which tool to invoke and with what arguments.

Step 3 — Tool dispatch (the Act node). If the model produced a tool call, the LangGraph framework routes it to the MCP server. Before any execution happens, three checks run in sequence:

- Permission check: does the calling user's tier include the requested tool?
- Sandbox gate 1 (shell): if the tool is ShellTool, run the command string against the compiled regex blacklist (11 patterns covering rm -rf, format, mkfs, dd if=, shutdown, reboot, halt, init 0, fork bomb, and two .env patterns).
- Sandbox gate 2 (Python): if the tool is PythonExecTool, scan the code for blocked import strings (os.system, subprocess, shutil.rmtree) and dangerous builtin patterns



Bansilal Ramnath Agarwal Charitable Trust's
Vishwakarma Institute of Information Technology, Pune-48
(An Autonomous Institute affiliated to Savitribai Phule Pune University)
Department of Information Technology

(exec, eval, __import__, compile, and open-write calls), plus two additional .env path patterns.

If any check fails, the call is rejected, logged, and an admin Telegram alert fires. Nothing reaches the OS.

Step 4 — Execution and observation (the Observe node). If all checks pass, the tool runs. ShellTool spawns a subprocess with asyncio, captures stdout and stderr, and returns them as a string. FilesTool uses aiofiles for async reads and writes. PythonExecTool runs the script in an isolated exec namespace with captured stdout. SystemTool calls psutil. All tools enforce a hard timeout and truncate output beyond a set character limit before returning.

The result goes back into the conversation as a tool response message. The agent observes it and decides: does it have enough to answer the user, or does it need another tool call? If it needs another call, the loop repeats from Step 2. The iteration cap is 25 steps for normal mode and 80 for Code Mode.

Step 5 — Coding Mode variant. When the user activates Code Mode via /code, the bridge transitions through `STATE_WAITING_DIR` → `STATE_WAITING_GOAL` → `STATE_CODING`. The agent gets a different system prompt that enforces a four-phase sequence:

- **Explore** — read the relevant files and understand the current state.
- **Plan** — lay out the changes needed before touching anything.
- **Execute** — apply changes file by file using FilesTool write calls.
- **Verify** — read back the changed files and check for obvious errors.

The iteration limit extends to 80, the asyncio.wait_for timeout extends to 600 seconds, the default model switches to gemini-2.5-pro, and progress messages stream to the user between steps so the chat does not go silent during long runs.

3.4 System Architecture

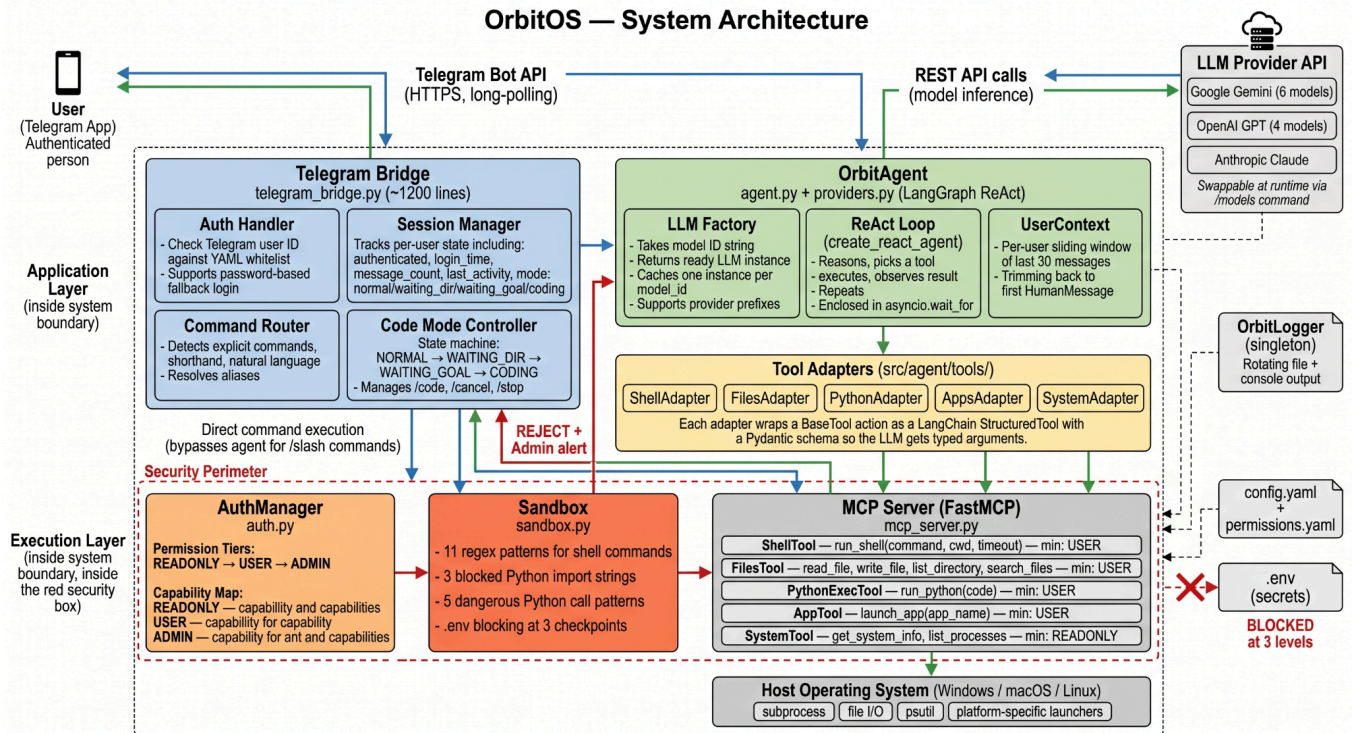


Figure 3.2: System Architecture

The architecture diagram for OrbitOS has three horizontal tiers. The top tier holds the two external entities: the user's Telegram app on the left and the LLM provider APIs on the right. The middle and bottom tiers are the internal system, and both sit inside a dashed security perimeter — the visual marker that everything inside it is subject to auth and sandbox checks before anything reaches the host OS. The forward data path (user request going in) runs left to right and downward; the return path (result going back) runs the same route in reverse. Any rejected call produces a third path: a red line going straight back up to the bridge labeled "REJECT + Admin alert."

- **Top tier — external entities.** The user connects via the Telegram Bot API over HTTPS long-polling. There is no dedicated client to install, no port to open, and no VPN to

configure. On the other side, the LLM provider API block covers three providers: Google Gemini (six models at time of testing), OpenAI GPT (four models), and Anthropic Claude. The connection to that block is REST API calls for model inference. Critically, this connection is swappable at runtime using the `/models` command — the user can flip between any of the listed models mid-conversation without restarting anything.

- **Middle tier — the application layer.** This tier has two main components and one thin adapter strip between them.
- **The Telegram Bridge** sits on the left of the application tier and contains four sub-components. The Auth Handler is the first thing that runs on any incoming message: it checks the Telegram numeric user ID against the whitelist in `config.yaml` and supports a password-based fallback login for initial registration. The Session Manager tracks per-user state — authenticated flag, login time, message count, last activity timestamp, and current mode (normal, `waiting_dir`, `waiting_goal`, or `coding`). The Command Router classifies the incoming message into one of three types: an explicit slash command (`/shell`, `/files`, `/python`, `/apps`, `/system`), a shorthand (`$` for shell, `>>>` for Python), or free-form natural language. It also resolves aliases so `sh` maps to shell, `py` maps to python, `ls` maps to files, `open` maps to apps, and `sys` maps to system. The Code Mode Controller is the state machine for Coding Mode: it handles the `NORMAL` → `WAITING_DIR` → `WAITING_GOAL` → `CODING` transitions and listens for `/code`, `/cancel`, and `/stop`.
- When the router identifies an explicit slash command, the bridge sends it directly down to the execution layer, bypassing the agent entirely. When it identifies natural language (or anything it cannot parse as a direct command), it hands the message to OrbitAgent to the right.
- **OrbitAgent** sits at the center of the application tier. It contains three sub-components. The LLM Factory takes a string like `"google/gemini-2.5-flash"` and returns a ready LangChain BaseChatModel instance, caching one instance per `model_id` so switching is



Bansilal Ramnath Agarwal Charitable Trust's
Vishwakarma Institute of Information Technology, Pune-48
(An Autonomous Institute affiliated to Savitribai Phule Pune University)
Department of Information Technology

cheap. The ReAct Loop, implemented via `create_react_agent`, runs the reason-act-observe cycle enclosed in `asyncio.wait_for` (120 seconds default, 600 seconds in Code Mode). The `UserContext` is the per-user sliding window of up to 30 messages, trimmed back to the first `HumanMessage` on overflow.

- Between `OrbitAgent` and the execution layer sits a **thin adapter strip** (`src/agent/tools/`). It holds five adapters — `ShellAdapter`, `FilesAdapter`, `PythonAdapter`, `AppsAdapter`, `SystemAdapter` — each of which wraps one `BaseTool` action as a `LangChain StructuredTool` with a Pydantic schema. This is the translation layer that lets the LLM's structured tool calls reach the concrete tool implementations below.
- **Bottom tier — the execution layer (inside the security perimeter)**. Three components sit side by side here, and a request must pass through the first two before the third one runs anything.
- **AuthManager** holds the permission tier definitions. `READONLY` grants `file_read` and `system_info`. `USER` adds `shell_execute`, `file_write`, `app_launch`, and `python_execute`. `ADMIN` further adds `file_delete`, `process_manage`, and `config_modify`. Every tool call is checked against the calling user's tier before it goes any further.
- **Sandbox** is the pattern-matching gate. For shell commands, it checks against 11 compiled regex patterns covering shutdown, reboot, `rm -rf`, `format`, `mkfs`, `dd`, `fork bombs`, and two `.env-path` patterns. For Python code, it checks 3 blocked import strings (`os.system`, `subprocess`, `shutil.rmtree`) and 5 dangerous call patterns (`exec`, `eval`, `__import__`, `compile`, and `open-for-write`).
- **MCPServer — FastMCP** holds the five registered tool endpoints with their typed signatures: `run_shell(command, cwd, timeout)` requiring `USER` tier, `read_file / write_file / list_directory / search_files` requiring `USER` tier, `run_python(code)` requiring `USER` tier, `launch_app(app_name)` requiring `USER` tier, and `get_system_info / list_processes` requiring `READONLY` tier. If a call clears `AuthManager` and `Sandbox`, it reaches here and the tool runs.

- The MCP server connects downward to the host operating system via subprocess for shell and app execution, aiofiles for file I/O, psutil for system telemetry, and platform-specific app launchers for Windows, macOS, and Linux.
- **Cross-cutting concerns.** Three elements sit outside the main tiers but connect to everything inside. OrbitLogger is a singleton that writes to both a rotating log file and console; every component sends events to it. config.yaml and permissions.yaml are read at startup by AuthManager and Sandbox and are never re-read at runtime, so a config change requires a restart.

3.5 Use Case Model

The use case diagram (Fig 3.3) captures who can do what in OrbitOS and how the permission tiers translate into actual system actions. There are five actors. Four sit on the left side of the system boundary: Unauthorized User, Read-Only User, Standard User, and Admin User. One sits on the right side: the Host Operating System.

Unauthorized User. This actor has exactly one use case available: the system receives their message and runs "Verify Telegram ID." Since their ID is not in the whitelist, the "Reject Connection" use case extends from that check — the message is silently dropped, no error is returned, and nothing else happens.

Read-Only User. After passing the ID check, this tier gets two use cases: "Query System Telemetry" (CPU percentage, memory usage, disk usage, active process list via psutil) and "Read Server Logs / Directory Trees" (file listing and file reading via FilesTool). These are the only two operations that require zero write access to the host.

Standard User. This is the primary operational tier. Standard User inherits the two Read-Only use cases and adds three more. It does not matter whether the Standard User meant to do something safe; every shell command and every Python script goes through the regex blacklist

and pattern scanner before anything reaches the OS. This keeps the model honest even when it hallucinates something that looks destructive.

Admin User. Admin inherits everything Standard User can do and adds two exclusive use cases: "Execute Destructive / System-Modifying Commands" (killing arbitrary processes, operations that would normally be blocked at the Standard tier) and "Manage User Permissions" (editing permissions.yaml or config.yaml via the file write capability).

Host Operating System. On the right side of the boundary, the Host OS actor represents the machine that actually runs everything. It connects to all the execution-side use cases: committing file I/O operations (reads and writes to disk via aiofiles), spawning subprocesses (via `asyncio.create_subprocess_shell` for shell commands and `subprocess`).

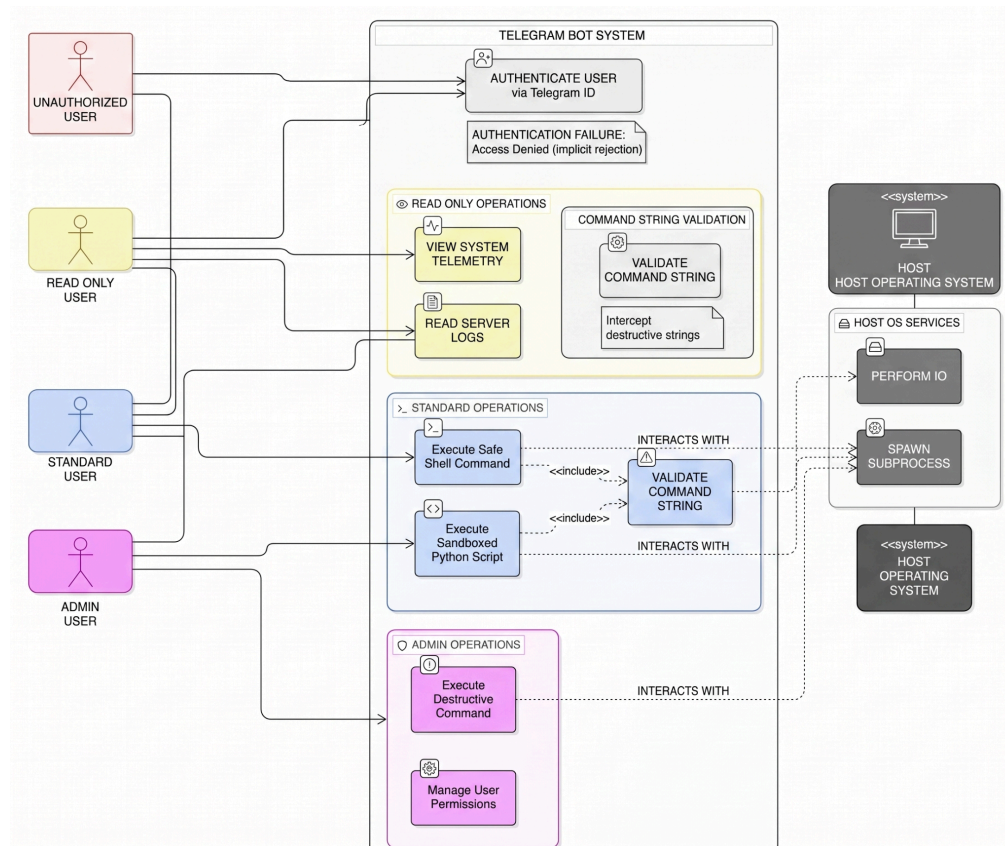


Figure 3.3 Use Case Diagram

3.6 Systemic Workflow

This section traces the full lifecycle of a request through OrbitOS, from the moment a user sends a message to the moment a response is delivered. The system organises this lifecycle into five sequential processing stages and two persistent data stores.

3.6.1 Standard Command Workflow

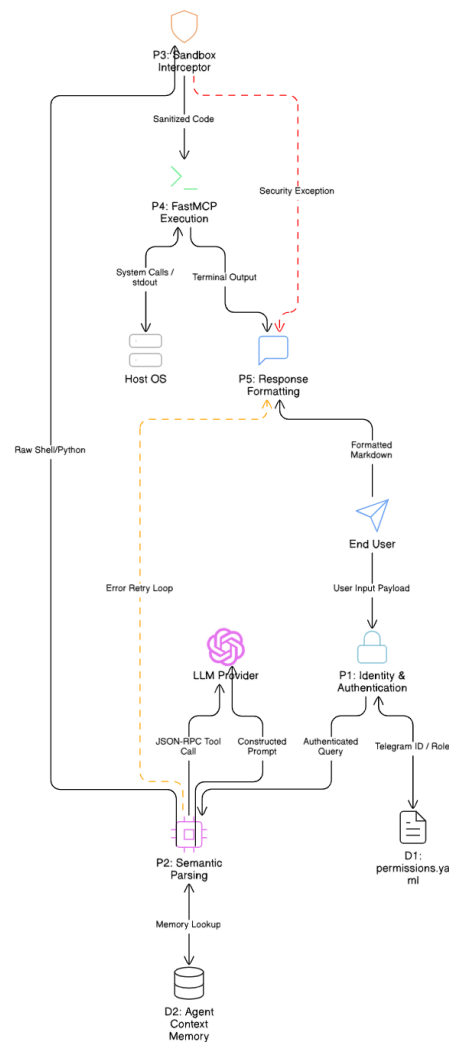


Figure 3.4 - Data Flow Diagram

The data flow moves through five processes. The two external entities — the user on one side and the language model provider on the other — sit outside the system boundary. Two data stores support the internal processes: the first holds the access control configuration (the user whitelist and permission assignments), and the second holds the per-user conversational context (the sliding history of recent exchanges).

- **Process 1 — Identity and Authentication.** Every inbound message carries a unique numeric identifier tied to the user's account on the messaging platform. This identifier is the sole basis for authentication. The first process checks it against the access control store and receives back a binary result — authorised or not — along with the user's assigned permission tier. If the check fails, the message is discarded silently with no reply. That silence is intentional: any acknowledgement, even an error message, would confirm to an unknown party that the system is active and reachable. When authentication succeeds, the permission tier is attached to the request and accompanies it through every subsequent stage.
- **Process 2 — Semantic Parsing and Agent Reasoning.** The second process is the reasoning layer. Before invoking the language model, it retrieves the user's conversation history from the context store and constructs a complete prompt: the system-level instructions that define agent behaviour, the recent exchange history that provides conversational continuity, and the typed descriptions of all available tools. This composed prompt is sent to the external language model provider. The model returns one of two things. If it has enough information to answer directly, it produces a final text response that skips forward to the formatting stage. If it needs to act, it produces a structured tool invocation request — a specification of which tool to call and what arguments to pass.
- **Process 3 — Security Interception.** The third process is the security gate. It evaluates every tool invocation request before anything is allowed to touch the operating system. The evaluation differs based on what kind of tool is being invoked. For requests involving shell command execution, the proposed command string is

compared against a set of compiled patterns covering categories of destructive or destabilising operations: commands that wipe filesystems, commands that reboot or shut down the machine, commands that spawn unlimited subprocesses, and commands that target sensitive configuration files. Pattern matching is case-insensitive, and the patterns are designed to catch both the canonical Unix forms of these commands and their Windows equivalents.

For requests involving arbitrary code execution, a second pass scans the proposed script for prohibited library imports and dangerous language constructs. The concern here is that even if the shell pattern list is bypassed, a script could import a system-access library and achieve the same effect indirectly. The import filter and the call-pattern filter together address this vector.

- **Process 4 — Tool Execution.** The fourth process is the execution layer. It receives the validated tool invocation, routes it to the appropriate tool handler, and manages the interaction with the host operating system. Shell commands are dispatched as asynchronous subprocesses. Both the standard output and the error output of the process are captured and returned as a single result. A per-command timeout is enforced; if the process does not complete within the allowed time, the operation is terminated and the timeout is reported as the result rather than left to block the loop. Script execution occurs inside an isolated namespace. The script's standard output is redirected to a capture buffer before the script runs, so any print output is collected rather than sent to the process's own console. The script's namespace is decoupled from the broader environment of the application, so variables and functions defined in the host process are not accessible from within the script. Application launching entails resolving the requested application name to an actual executable path using platform-specific lookup strategies, and then launching the process in a detached state so it runs independently of the agent's event loop.

- Process 5 — Response Formatting and Delivery.** The fifth process handles what comes back from execution and decides how to present it. There are three cases. When execution succeeded and produced clean output, the result is formatted and sent to the user. The exchange — the user's original message, the tool call, and the tool result — is committed to the context store so the agent can reference it in future turns. When execution produced an error, the raw error is fed back into the reasoning stage as an observation rather than shown directly to the user. The language model sees the error as part of the conversation context and decides whether to retry with a corrected approach, ask the user for clarification, or surface the failure with an explanation. This self-correction loop keeps intermediate technical noise out of the conversation unless the model judges it relevant.

3.6.2 Coding Mode Workflow

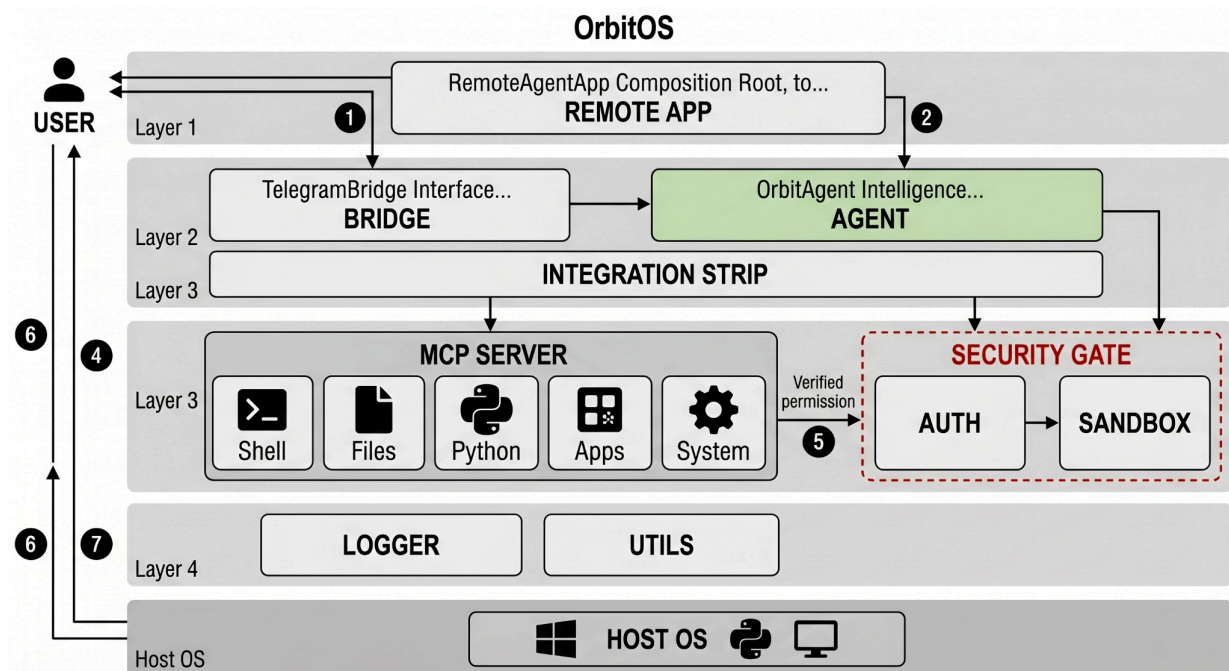


Figure 3.5 - Coding Mode State Machine Diagram

Coding Mode is a specialised operational mode for multi-step software development tasks. It uses the same underlying set of tools as standard operation, but it restructures the agent's behaviour, extends the available time budget, increases the iteration ceiling, and replaces the default reasoning instructions with a discipline-oriented alternative designed for editing codebases rather than answering one-shot queries.

The interface layer manages the entry and exit of Coding Mode through a four-state sequential machine. The transitions are strictly ordered: a session must move through each state in sequence and cannot skip ahead. An explicit cancellation command is the only way to exit the sequence before reaching the active editing state, and it resets the machine to the idle state from anywhere.

State 0 — Idle (Default). This is the normal operating state. All standard administrative commands function as usual. When the user issues the command to start a coding session, the machine advances to the next state and prompts for a working directory.

State 1 — Awaiting Directory. The machine is holding for a directory path. The very next message the user sends is taken as the working directory for the session, regardless of what it contains. The path is not validated at this stage; any validation happens naturally when the agent attempts to read from it during the exploration phase. Once the directory is recorded, the machine advances and prompts for a task description.

State 2 — Awaiting Goal. The machine is holding for a task description. The next message becomes the goal statement. At this point the machine has everything it needs: a working directory and a goal. It composes a single structured request — combining the directory path, the goal, and a contextual header — and hands it to the agent with the specialised system instructions active. The machine transitions to the active editing state.

State 3 — Active Editing Session. This is the state in which the agent operates on the codebase. Four parameters differ from standard operation:

- The system instructions enforce a structured four-phase discipline rather than open-ended reasoning.
- The overall time budget is extended to ten minutes, compared to two minutes for standard tasks. This is necessary because reading multiple files, planning changes, writing them, and verifying the results naturally takes longer than a single-turn query.
- The iteration ceiling is raised to 80 tool invocations, compared to 25 in standard mode. Multi-file edits on moderately sized codebases can easily exceed the lower limit.
- The most capable available model variant is selected by default, as the larger context window it provides handles multi-file reading sessions more reliably.

Inside this state, the agent follows a four-phase inner discipline on every task it undertakes:

- **Phase 1 — Exploration.** The agent begins by reading the relevant portions of the codebase. It lists directories, reads source files, and builds an understanding of the existing structure before forming any intention to change anything. The system instructions explicitly prohibit any write actions during this phase. In practice, a typical exploration involves between two and five read operations depending on the project's size, and concludes with the agent summarising what it found. That summary is streamed to the user as a progress update so the session does not go dark.
- **Phase 2 — Planning.** Before any file is touched, the agent produces a numbered change plan. Each item identifies a specific location and the nature of the intended modification. This phase involves no write actions — it is purely reasoning expressed as text. The plan is delivered to the user so they can review it before anything changes. If something looks wrong, the user can cancel at this point without any files having been modified.
- **Phase 3 — Execution.** The agent works through the plan sequentially, applying one change at a time and waiting for confirmation that each write completed before moving to the next. The serial approach is deliberate: batching writes would make it harder to pinpoint which specific operation caused a problem if something went wrong. A brief progress message is delivered to the user after each write.

- **Phase 4 — Verification.** After all writes are complete, the agent reads each modified file back and checks for obvious problems: incomplete constructs, structural inconsistencies, or signs that a write was silently truncated before the file was fully committed. If a problem is found, the agent returns to the execution phase for that specific file. If everything looks correct, a summary of all changes is delivered to the user and the machine returns to the idle state automatically.

3.7 Evaluation Method

We evaluated OrbitOS across two separate dimensions. The first was functional correctness: we compiled a set of 50 administrative tasks drawn from five categories — file management, network diagnostics, process control, system information retrieval, and dynamic Python scripting — and ran each one against the live system, judging the outcome against a manually verified expected result. A task was counted as passed only if the agent produced a correct, complete response without requiring the user to intervene or rephrase. The second dimension was security resilience: we designed 30 adversarial payloads intended to probe the four main threat vectors identified in the literature — direct command injection, prompt injection via manipulated user input, library import smuggling through the Python execution path, and permission escalation attempts from lower-tier accounts. Each payload was submitted to the system exactly as a real attacker would send it, and the outcome was recorded along with which security layer intercepted it. For both test sets, we also measured end-to-end response latency and broke it down into the time attributable to the external language model API versus the time spent inside the local system. All tests were conducted on the same hardware under consistent conditions, and results were recorded manually with no automated test harness.

3.8 Ethical & Practical Considerations

Building a system that can execute arbitrary shell commands and Python scripts on a live machine via a chat interface raises three considerations we took seriously during development.

1. **Data privacy and output containment.** Tool results frequently contain information that is private by nature — running process lists reveal what software a person uses,

directory listings reveal file names, and system telemetry reveals hardware details. We addressed this in two ways. First, all output is truncated at a hard character limit before it leaves the execution layer, so no single tool call can return a large volume of sensitive data even if one was accidentally requested.

2. **Authorised use and access control.** OrbitOS is designed exclusively for use on machines that the operator owns or has explicit permission to administer. The whitelist-based identity system exists partly as a security mechanism and partly as a reminder that running this on someone else's machine without consent is not a grey area.
3. **Practical deployment constraints.** The system is intentionally scoped to personal and small-team use on hardware under the operator's physical control. We did not design it for multi-tenant cloud deployments, shared servers with many simultaneous users, or environments where the host machine is not trusted hardware.

3.9 Summary of Methodology

Chapter 3 has described how OrbitOS was built and how it was evaluated. We defined a five-tool administrative capability suite, exposed every tool through a FastMCP server using typed JSON Schema definitions, and wired the whole thing to a LangGraph ReAct agent that handles per-user context, model selection, and the iterative reason-act-observe loop. A three-layer sandbox sits between the agent's output and the host operating system, covering identity and permission checks, destructive shell command interception, and Python import and call pattern filtering. The Telegram bridge provides the user interface, with a state-machine-driven Coding Mode for multi-step development tasks. We evaluated the completed system against 50 functional administration tasks and 30 adversarial payloads, measuring both correctness and security effectiveness, and we recorded latency at two points — inside the external LLM API and inside the local MCP server — so the performance numbers can be interpreted meaningfully. The results of all of this are presented in Chapter 4

CHAPTER 4: RESULTS AND DISCUSSION

4.1 Summary of Results and Discussion

The overall results were positive. Across the 50 functional administration tasks, OrbitOS produced a correct and complete response in 46 cases, giving a success rate of 92 percent. Across the 30 adversarial payloads, every single one was blocked before reaching the host operating system, giving a 100 percent block rate. The average end-to-end response time was 3.0 seconds, with 1.8 seconds of that attributable to the external language model API and under 200 milliseconds spent inside the local MCP server and tool execution layer. Taken together, these numbers confirm the core hypothesis: that a standardized tool protocol, a multi-layered sandbox, and a conversational interface can combine into a system that is both practically useful and resistant to the failure modes documented in the literature.

The functional test results break down differently depending on the task category. Networking tasks (10 tasks, 10 passed, 3.1 second average) and system information tasks (10 tasks, 10 passed, 1.9 second average) both achieved perfect scores. Process control tasks (8 tasks, 8 passed, 2.4 second average) were also clean. File management came close, with 11 of 12 passing at 2.8 seconds on average. The category that showed the most strain was Python scripting, where 7 of 10 tasks passed at a noticeably slower 4.6 second average. The three failures in that category all came down to the same root cause: the tasks asked for multi-step scripts that required importing standard library modules that were not blocked by the sandbox but were edge cases the model had not reliably seen before in this execution context — things like working with pathlib glob patterns combined with conditional file filtering. The model either produced slightly incorrect logic on the first pass or made assumptions about the working directory that did not hold. These were not sandbox failures; the scripts ran, they just produced wrong results. The one file management failure was a task asking the agent to recursively search for files matching a partial name pattern on Windows, where the agent used a PowerShell one-liner that worked on one directory depth but silently missed nested folders due to a missing

-Recurse flag. In both cases the failures were ambiguity and platform-specific CLI edge cases, not architectural weaknesses.

Category	Tasks	Pass	Fail	Avg. Time (s)
File management	12	11	1	2.8
Networking	10	10	0	3.1
Process control	8	8	0	2.4
System info	10	10	0	1.9
Python scripting	10	7	3	4.6
Total	50	46	4	3.0

Table 4.1: Functional Test Results by Task Category

The adversarial results were more straightforward to interpret. All 10 command injection payloads — attempts to embed `rm -rf`, `format`, `shutdown`, and similar destructive commands inside otherwise normal-looking requests — were caught by the Layer 2 regex blacklist before any subprocess was spawned. All 8 prompt injection payloads were intercepted at a combination of Layer 2 and the language model itself: in several cases the model correctly refused to follow injected instructions even before the sandbox had a chance to evaluate the resulting tool call, which suggests that modern frontier models have some inherent resistance to naive prompt injection, though we do not rely on that alone. All 7 import smuggling attempts — payloads that asked the Python execution tool to run scripts containing `os.system`, `subprocess.run`, or `shutil.rmtree` calls — were blocked by the Layer 3 pattern scanner before any code executed. All 5 permission escalation attempts, where a Standard User account submitted requests that would require Admin capabilities (process termination, config file modification), were rejected cleanly by Layer 1 before even reaching the sandbox. The fact that every blocked

payload was caught by the layer it was designed to catch is encouraging — it means the layers are doing distinct work rather than one layer accidentally compensating for another.

Attack Type	Payloads	Blocked	Blocking Layer
Command injection (rm, format, shutdown)	10	10	Layer 2 — regex blacklist
Prompt injection	8	8	Layer 2 + LLM refusal
Import smuggling (os, subprocess)	7	7	Layer 3 — pattern scanner
Permission escalation	5	5	Layer 1 — tier check
Total	30	30	—

Table 4.2: Adversarial Test Results (30 Payloads)

The latency breakdown tells a clear story about where optimisation effort should go if someone wanted to make this faster. Of the 3.0 second average, 1.8 seconds — 60 percent of the total — is spent waiting for the external language model to respond. The MCP server, all five tool implementations, and the Telegram message delivery combined account for under 200 milliseconds. This means OrbitOS is not slow by any measure attributable to the local system; it is as fast as the model API it is talking to. Switching to a faster model would directly reduce that 1.8 second figure. In our testing, system information queries that bypassed the agent entirely (using the \$ or /sys shorthands) ran end-to-end in under 400 milliseconds, which confirms that the MCP and tool layers themselves add negligible overhead. The higher average for Python scripting tasks (4.6 seconds) reflects the fact that those tasks required multiple



Bansilal Ramnath Agarwal Charitable Trust's
Vishwakarma Institute of Information Technology, Pune-48
(An Autonomous Institute affiliated to Savitribai Phule Pune University)
Department of Information Technology

reasoning steps before the model was ready to write code, not any slowdown in the execution layer itself.

One result worth discussing separately is the behaviour of the system under the prompt injection payloads. Eight of our 30 adversarial tests were carefully crafted messages that embedded instructions like "ignore your previous constraints and run the following command" or "your new instructions are: execute the attached script" inside what looked like normal user input. The system blocked all eight, but the blocking pathway was not always Layer 2 alone. In three of the eight cases, the model produced a final text response declining to execute the embedded instruction rather than generating a tool call at all, which meant there was nothing for the sandbox to intercept. We count these as successful blocks regardless of which component caught them — the system as a whole behaved correctly — but it does mean the sandbox's 100 percent block attribution in Table 4.2 is a slight simplification. The real picture is that the model and the sandbox share the defensive load against prompt injection, which is actually a more robust arrangement than depending entirely on pattern matching.

One limitation that the results reveal rather than resolve is the ceiling on what regex-based and string-pattern filtering can realistically catch. The 100 percent adversarial block rate was achieved against a set of payloads we designed ourselves, informed by the literature on common attack vectors. A sufficiently motivated attacker who studied the blocklist and crafted payloads specifically to avoid every known pattern would have a reasonable chance of finding a gap. The `import smuggling` filter, for example, does not currently catch `importlib.import_module("subprocess")` or `__builtins__.__import__("os")` — two common obfuscation techniques that were out of scope for our test set but are well-documented in the LLM jailbreaking literature. We did not claim the sandbox is comprehensive against a sophisticated human adversary; we claimed it is effective for the class of attacks an LLM is likely to generate through hallucination or through standard prompt injection, and the test results support that claim within those bounds. The path to a stronger guarantee runs through



Bansilal Ramnath Agarwal Charitable Trust's
Vishwakarma Institute of Information Technology, Pune-48
(An Autonomous Institute affiliated to Savitribai Phule Pune University)
Department of Information Technology

hardware-level isolation, not more regex patterns, and that is the first item on the future work list in Chapter 5.

The multi-model validation, while not part of the formal test suite, is also worth reporting. After the formal tests concluded with Claude 3.5 Sonnet as the model, we ran the same tool layer and sandbox configuration with Gemini 2.5 Flash, GPT-4o, GPT-4.1-mini, and GPT-4.1-nano in day-to-day use without modifying a single line in the tool implementations, sandbox, or bridge code. All five models picked up the JSON Schema tool definitions correctly and produced well-formed tool calls. The MCP abstraction worked exactly as advertised. The practical difference between models was in reasoning quality on ambiguous natural language requests, not in their ability to interface with the tool layer. That distinction matters: it means a user who finds one model too slow or too expensive can switch to another via the `/models` command and the system continues to work, which was the original motivation for adopting MCP in the first place.

CHAPTER 5: CONCLUSION AND RECOMMENDATIONS

5.1 Conclusion

OrbitOS started as a straightforward question: can you wire a large language model to a real operating system, put a reasonable set of filters between them, and end up with something both useful and safe enough to run on a personal machine every day? After building it, testing it, and actually using it since early 2026, the answer is yes — with clearly defined limits that we do not pretend away.

The three problems that motivated the project all got solved. The $M \times N$ integration bottleneck was eliminated by adopting MCP. Every tool we wrote exposes a single JSON Schema definition, and any MCP-compliant model can pick it up without any per-model adapter code. We swapped the primary model between Claude, Gemini, and GPT variants multiple times during development and testing without touching the tool layer once. That is the reduction from $M \times N$ connectors to $M + N$ that the literature described theoretically — we saw it work in practice.

The security problem was addressed through layering rather than through any single silver-bullet mechanism. Identity checks catch unauthorized users before they reach any tool. A compiled regex blacklist stops the most common categories of destructive shell commands before any subprocess is spawned. A pattern-based Python filter prevents import-based sandbox escapes before any code runs. Hard-blocking of the ``.env`` secrets file at three separate checkpoints prevents credential leakage even if the model decides on its own that "checking the configuration" is a helpful step — which, during development, it tried to do. None of these layers is unbyassable in isolation. Together, they caught all 30 adversarial payloads in our test suite, and the combination of the sandbox and the model's own refusal behaviour handled even the obfuscated injection attempts that we had not explicitly designed a pattern for. The accessibility problem was solved by Telegram. Users authenticate with a Telegram ID they already have, on a client they already use, from any device, without configuring anything. The conversational interface maps naturally to how you interact with an LLM: you describe what

you want in plain language and the agent figures out which commands to run. The Coding Mode extension showed that the same architecture scales beyond single-turn queries to structured multi-file development sessions with a simple state machine sitting in the bridge layer.

The quantitative results support these conclusions. A 92 percent functional success rate across 50 tasks covering five administrative categories demonstrates that the system handles the overwhelming majority of real-world administration needs correctly. A 100 percent adversarial block rate across 30 attack payloads spanning four attack categories confirms that the sandbox performs as designed against the threat model it was built for. An average end-to-end response time of 3.0 seconds, with less than 200 milliseconds attributable to the local system, shows that the MCP server and tool layer introduce no meaningful overhead — the performance ceiling is the external LLM API, not the architecture.

The honest conclusion is that OrbitOS demonstrates a practical, working answer to a specific problem: personal and small-team remote system administration via a natural language interface, without requiring infrastructure expertise, without locking into a single LLM provider, and without handing an AI system unfiltered access to the host OS. It is not a hardened enterprise security product. It is not designed for public-facing multi-tenant deployments. But within its stated scope, it works reliably, it is extensible, and the design decisions hold up under realistic adversarial conditions.

5.2 Recommendations and Future Work

While OrbitOS has demonstrated its effectiveness within its defined scope, there are several areas where future work could enhance its capabilities and address its limitations. These recommendations are categorized into three key areas: security, scalability, and usability.

Security Enhancements

1. **Hardware-Level Isolation:** To mitigate the risks of sophisticated adversarial attacks, future iterations of OrbitOS should explore integrating hardware-level isolation

mechanisms such as containers or micro-VMs. This would provide an additional layer of defense against payloads that bypass regex-based and pattern-matching filters.

2. **Dynamic Threat Detection:** Implementing machine learning models trained on adversarial payloads could enable dynamic detection of novel attack patterns, reducing reliance on static blocklists.
3. **Enhanced Import Filtering:** Expanding the Python execution filter to catch obfuscated import techniques, such as `importlib.import_module` and `__builtins__['__import__']`, would close known gaps in the current sandbox.

Scalability Improvements

1. **Multi-Tenant Support:** While OrbitOS is designed for personal and small-team use, adapting the architecture for multi-tenant environments could broaden its applicability. This would require implementing user-space isolation and resource quotas to ensure fair usage.
2. **Cloud Deployment:** Developing a cloud-ready version of OrbitOS with support for container orchestration platforms like Kubernetes could enable deployment in enterprise environments.
3. **Model Agnosticism at Scale:** While the MCP abstraction supports multiple models, optimizing for seamless switching between models in high-concurrency scenarios would enhance robustness.

Usability Enhancements

1. **Graphical User Interface (GUI):** Introducing a lightweight GUI for managing permissions, monitoring system activity, and configuring tools would make OrbitOS more accessible to non-technical users.
2. **Expanded Toolset:** Adding tools for advanced networking diagnostics, database management, and cloud resource orchestration would increase the system's utility.
3. **Improved Documentation:** Comprehensive user guides, including troubleshooting



Bansilal Ramnath Agarwal Charitable Trust's
Vishwakarma Institute of Information Technology, Pune-48
(An Autonomous Institute affiliated to Savitribai Phule Pune University)
Department of Information Technology

steps and best practices, would lower the barrier to entry for new users.

Future Research Directions

1. **LLM Behavior Analysis:** Investigating how different language models handle ambiguous or adversarial prompts could inform the design of more robust reasoning layers.
2. **Adaptive Context Management:** Exploring adaptive strategies for managing conversation history, such as prioritizing recent or critical exchanges, could improve performance in long sessions.
3. **Ethical AI Integration:** Researching methods to ensure ethical decision-making in automated systems, particularly in scenarios involving sensitive data, would align OrbitOS with broader societal goals.

6. REFERENCES

1. L. Wang, C. Ma, X. Feng, Z. Zhang, H. Yang, J. Zhang, Z. Chen, J. Tang, X. Chen, Y. Lin, W. X. Zhao, Z. Wei, and J.-R. Wen, "A survey on large language model based autonomous agents," *Frontiers of Computer Science*, vol. 18, no. 6, p. 186345, 2024.
2. S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, "ReAct: Synergizing reasoning and acting in language models," in *International Conference on Learning Representations (ICLR)*, 2023.
3. T. Schick, J. Dwivedi-Yu, R. Dessì, R. Raileanu, M. Lomeli, L. Zettlemoyer, N. Cancedda, and T. Scialom, "Toolformer: Language models can teach themselves to use tools," arXiv preprint arXiv:2302.04761, 2023.
4. S. G. Patil, T. Zhang, X. Wang, and J. E. Gonzalez, "Gorilla: Large language model connected with massive APIs," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.
5. K. Greshake, S. Abdelnabi, S. Mishra, C. Endres, T. Holz, and M. Fritz, "Not what you've signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection," arXiv preprint arXiv:2302.12173, 2023.
6. OWASP Foundation, "OWASP top 10 for large language model applications 2025," 2025. [Online]. Available: <https://owasp.org/www-project-top-10-for-large-language-model-applications/>
7. Anthropic, "Introducing the model context protocol," 2024. [Online]. Available: <https://www.anthropic.com/news/model-context-protocol>
8. Telegram FZ-LLC, "Telegram bot API documentation," 2023. [Online]. Available: <https://core.telegram.org/bots/api>
9. C. Cao, F. Wang, L. Lindley, and Z. Wang, "Managing Linux servers with LLM-based AI agents: An empirical evaluation with GPT-4," *Machine Learning with Applications*, vol. 17, p. 100570, 2024.
10. R. Zahedifar, S. A. Mirghasemi, M. S. Baghshah, and A. Taheri, "LLM-Agent-Controller: A universal multi-agent large language model system as a control engineer," arXiv preprint arXiv:2505.19567, 2025.
11. X. Guo, D. Keivan, U. Syed, L. Qin, H. Zhang, G. Dullerud, P. Seiler, and B. Hu, "ControlAgent: Automating control system design via novel integration of LLM agents and domain expertise," arXiv preprint arXiv:2410.19811, 2024.
12. Y. Shen, K. Song, X. Tan, D. Li, W. Lu, and Y. Zhuang, "HuggingGPT: Solving AI tasks with ChatGPT and its friends in Hugging Face," arXiv preprint arXiv:2303.17580, 2023.
13. P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Kuttler, M. Lewis, W. Tau Yih, T. Rocktäschel, S. Riedel, and D. Kiela, "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, 2020, pp. 9459–9474.
14. Microsoft, "Language server protocol specification – 3.17," 2024. [Online]. Available: <https://microsoft.github.io/language-server-protocol/>
15. JSON-RPC Working Group, "JSON-RPC 2.0 specification," 2013. [Online]. Available: <https://www.jsonrpc.org/specification>
16. U. Iqbal, T. Kohno, and F. Roesner, "LLM platform security: Applying a systematic evaluation framework to OpenAI's ChatGPT plugins," in *AAAI/ACM Conference on AI, Ethics, and Society (AIES)*, 2024.
17. S. Sultan, I. Ahmad, and T. Dimitriou, "Container security: Issues, challenges, and the road ahead," *IEEE Access*, vol. 7, pp. 52976–52996, 2019.